Interakcija čovjeka i robota: analiza poze i gesta

Tara Knežević,
mag. ing. mech.

Vježba 1

Body pose estimation, MediaPipe

Mediapipe:

- biblioteka za strojno učenje (*Google Research)*
- Koristi se za obradu i analizu medijskih podataka kao što su video, audio i slike u stvarnom vremenu

Google Research

MediaPipe

Mediapipe dependencies:

- Pri radu s MediaPipe bibliotekom, zbog velikog broja ovisnosti koristiti ćemo virtualno okruženje (numpy, scipy, protobuf, jax, matplotlib, opencv...)
- Mediapipe ima gotove wheelove za Python 3.9–3.12
- **Provjereno radi:** Python **3.10 64-bit** + MediaPipe **0.10.x** na Windowsu

Mediapipe modaliteti:

1. **MediaPipe Hands** – detekcija i praćenje šaka
2. **MediaPipe Face Mesh** – detekcija lica i ključnih točaka lica
3. **MediaPipe Pose** – praćenje cjelokupnog tijela i ključnih točaka tijela
4. MediaPipe Objectron – 3D detekcija objekata
5. MediaPipe Holistic - kombinirano praćenje lica, ruku i tijela
6. MediaPipe Solutions – kolekcija unaprijed izgrađenih rješenja i primjera koji pokrivaju razne slučajeve upotrebe

MediaPipe – što sve možemo?

1. Praćenje ruku, lica i tijela – detekcija i praćenje ruku, lica i tijela te njihovih ključnih točaka
2. Segmentacija tijela – prepoznavanje različitih dijelova tijela u slici ili videozapisu
3. Detekcija objekata – identifikacija i praćenje objekata u stvarnom vremenu
4. Označavanje i praćenje kretanja – praćenje kretanja kroz prostor i vrijeme (u videozapisima)

Mediapipe prerequisites:

Instalacija Pythona 3.10 na Windowsu

- https://www.python.org/downloads/ - preuzeti **“Windows installer (64-bit)”** za Python 3.10.x.
- Nakon preuzimanja datoteke pokrenuti instalaciju (označiti **“Add Python 3.10 to PATH”**)
- Provjera nakon instalacije u terminalu: python --version

```
C:\Users\taran>python --version
Python 3.10.11
```

Virtualno okruženje:

- Alat koji omogućava stvaranje izoliranih okruženja za projekte
- venv = *privatni Python sandbox* za svaki projekt

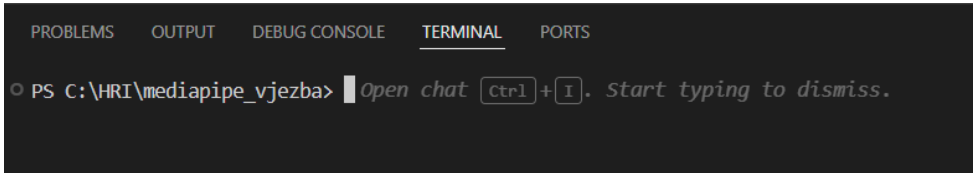
1. Izolacija projekata: Omogućava svakom projektu da koristi vlastite verzije knjižnica, što sprečava konflikte među projektima
2. Kontrola nad ovisnostima: Olakšava upravljanje i održavanje ovisnosti specifičnih za projekt
3. Reproducibilnost: Pomaže u reproduciranju okruženja na različitim strojevima ili među različitim članovima tima

Korištenje virtualnog okruženja:

- Problem: “Could not install... long path” - radite projekt u kratkim putanjama
- Kreiranjem novog direktorija koji sadrži **kopiju Python interpretera** i pip-a, zajedno s vlastitom **kopijom knjižnica** i skripti potrebnih za projekt
- Radni tijek rada s virtualnim okruženjem:
  1. Kreiranje i aktivacija virtualnog okruženja
  2. Instalacija potrebnih knjižnica te rad na projektu
  3. Deaktivacija virtualnog okruženja
  4. Kod ponovnog rada na projektu, reaktivacija virtualnog okruženja

Kreiranje virtualnog okruženja:

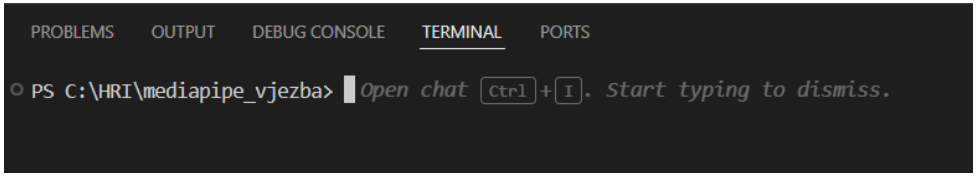
1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter

Kreiranje virtualnog okruženja:

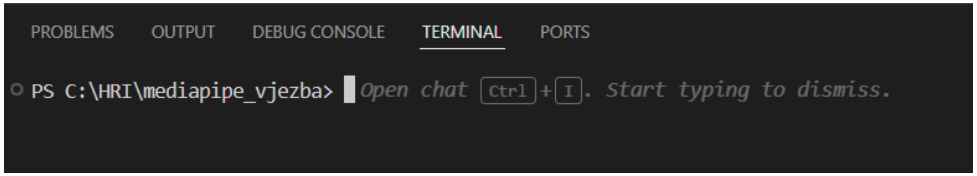
1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter

Kreiranje virtualnog okruženja:

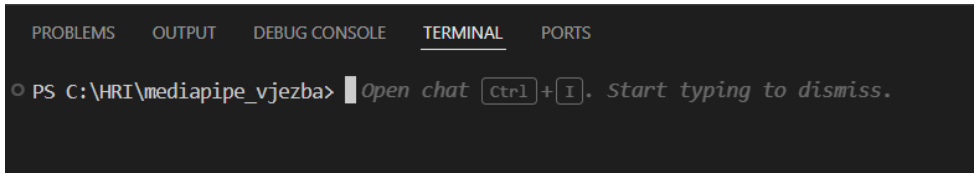
1. Otvorite PowerShell/CMD u folderu projekta:



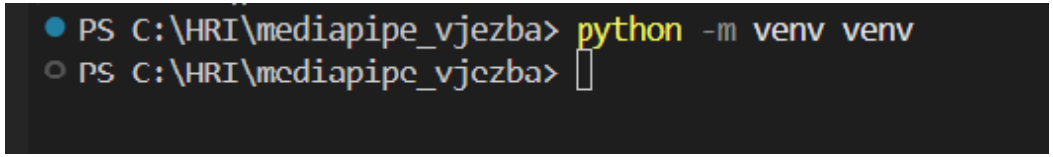
2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter
   - -m venv – kaže Pythonu: “napravi virtualno okruženje”
   - venv – ime foldera u kojem će ga napraviti

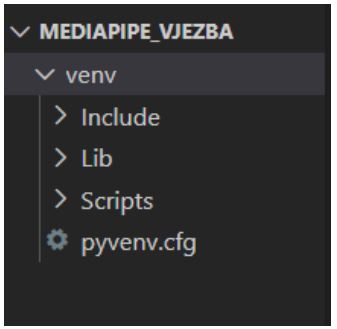
Kreiranje virtualnog okruženja:

1. Otvorite PowerShell/CMD u folderu projekta:



2. Kreirajte venv: *python -m venv venv*
   - python – pokreće Python interpreter
   - -m venv – kaže Pythonu: “napravi virtualno okruženje”
   - venv – ime foldera u kojem će ga napraviti





- Uspješno kreiranje virtualnog okruženja – kreiran je folder ‘venv’

Kreiranje virtualnog okruženja:

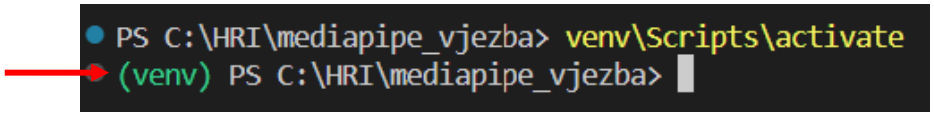
U slučaju da je na računalu instalirano više verzija Pythona

1. Provjera koje verzije postoje: py -0p
2. Kreiranje venv-a s konkretnom verzijom: py -3.10 -m venv venv
3. (Ako py ne radi, doći preko putanje):

   "C:\Users\...\Python310\python.exe" -m venv venv

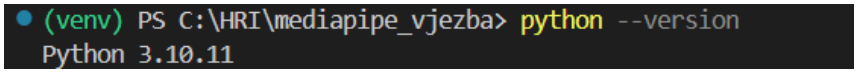
Aktivacija i deaktivacija virtualnog okruženja:

1. Aktivacija (Windows): *venv\Scripts\activate*

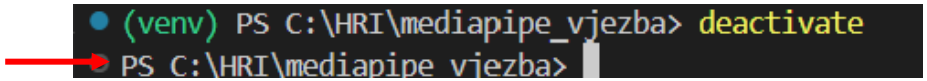


- Uspješna aktivacija virtualnog okruženja – **(venv)** ispred putanje

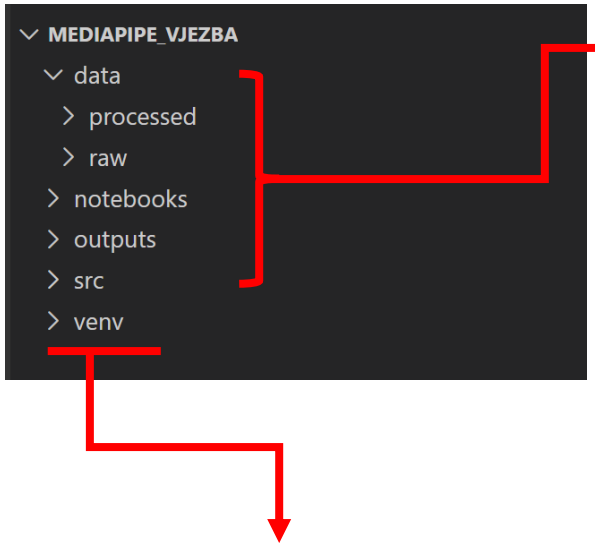
2. Provjera: *python –version*



3. Deaktivacija: *deactivate*



Struktura projekta unutar virtualnog okruženja:



SAMI kreiramo:

1. src - naš Python kod
2. data – podaci
    1. processed – rezultati
    2. raw – ulazni podaci
3. outputs - slike grafova, logovi, modeli, izvještaji
4. notebooks - Jupyter notebooks

Automatski generirano pri izradi virtualnog okruženja

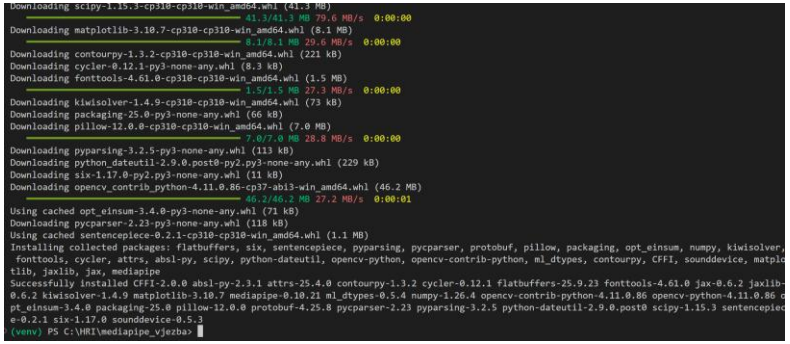
- “venv” ostaje u rootu i kod se nikad ne stavlja u venv

MediaPipe instalacija:

Unutar venv

- Pretpostavka: virtualno okruženje je aktivno

1. pip: python.exe -m pip install --upgrade pip
2. MediaPipe: pip install mediapipe opencv-python
3. Provjera:
   - python
   - import mediapipe as mp
   - print(mp.__version__)




```
(venv) PS C:\HRI\mediapipe_vjezba> python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import mediapipe as mp
>>> print(mp.__version__)
0.10.21
```

Korištenje virtualnog okruženja:

- Preduvjet: aktivirano virtualno okruženje
- Pozicionirati se u projektni folder unutar terminala
- Pokretanje koda: python src/neki_kod.py

Zadatak 0:

Priprema virtualnog okruženja i instalacija MediaPipe biblioteke

- Prema uputama s prezentacije potrebno je stvoriti vlastito virtualno okruženje
- Prema uputama s prezentacije potrebno je instalirati MediaPipe datoteku

- Provjere:
  - Verzija Pythona
  - Verzija MediaPipe-a

Intel RealSense kamera:

- Dubinska + RGB kamera (dvostruki senzor)
- Daje nam:
  - RGB sliku (kao obična kamera)
  - Depth mapu (udaljenost svakog piksela od kamere)



INTEL REALSENSE



FRAMOS

Intel RealSense instalacija:

1. Instalirati Intel RealSense SDK 2.0 za Windows
   - https://github.com/realsenseai/librealsense/releases/tag/v2.57.4
   - Instalirati **Intel.RealSense.SDK-WIN10-2.57.4.9699.exe** i **Intel.RealSense.Viewer.exe**
   - **NE** u venv - driver je Windows system-level komponenta koja omogućava OS-u da uopće vidi kameru kao uređaj
   - Komponente koje će se instalirati: driver, firmware tools, RealSense Viewer

Intel RealSense instalacija:

2. Instaliraj Python binding u virtualno okruženje
    - **Unutar** aktivnog venv: pip install pyrealsense2
    - Provjera:
        - python
        - import pyrealsense2 as rs
        - print(rs.__file__) - kaže put odakle je modul učitan

Priprema podataka za obradu:

- S e-učenja skinuti mapu „1137.zip” → unzipati → premjestiti mapu u: projekt/data/bpe/raw
- S e-učenja skinuti mapu „src.zip” → unzipati → premjestiti mapu u: projekt/src/bpe

Primjeri vježba1:

- 01_load_image.py - učitavanje i prikaz slike (OpenCV)
- 02_realsense_feed.py – RealSense feed
- 03_find_keypoints.py - MediaPipe Pose + skeleton na slici
- 04_get_keypoints_data.py - čitanje i ispis odabrane ključne točke na 2D slici
- 05_get_keypoints_data_RS.py - čitanje i ispis odabrane ključne točke na RealSense feed-u
- 06_landmarks_list.py – ispis svih ključnih točaka dostupnih na Pose modulu
- 07_angle_calculation.py - funkcija za kut + kut u laktu
- 08_reading_data_pennaction .py - čitanje .mat datoteke i pregled sadržaja
- 09_reading_data_pennaction_xy .py - izvlačenje X,Y za prvi frame

Primjer - Učitavanje slike i prikaz u OpenCV-u:

- OpenCV radi sa slikama kao s matricama piksela (numpy array)
- cv2.imread() učitava sliku iz datoteke
- cv2.imshow() prikazuje sliku u prozoru
- Ovo je temelj za obradu slika bez kamere/real-time-a
- Kod 01_load_image.py
  - Napomena: provjeriti putanju do slike

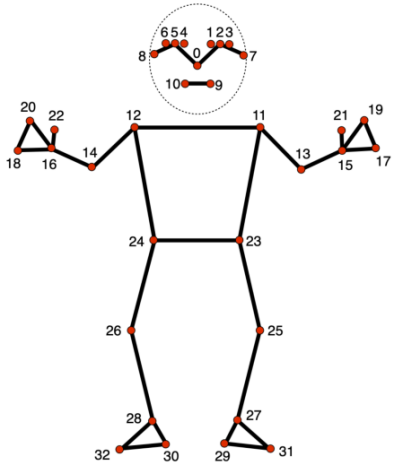
Primjer – RealSense live feed:

- Body pose model mora obrađivati slike frame-by-frame
- LIVE FEED znači:
  - Stalno čitanje nove slike
  - Detekcija
  - Prikaz rezultata

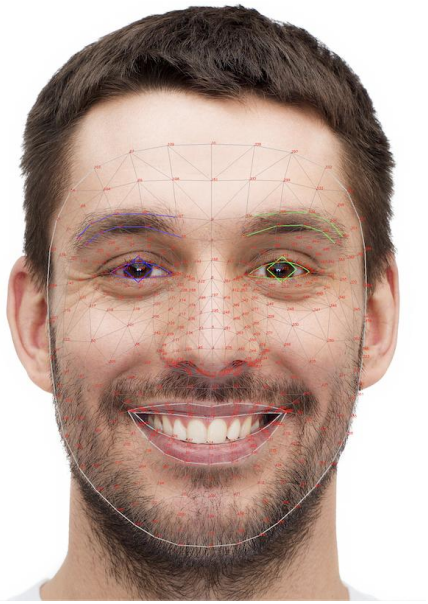


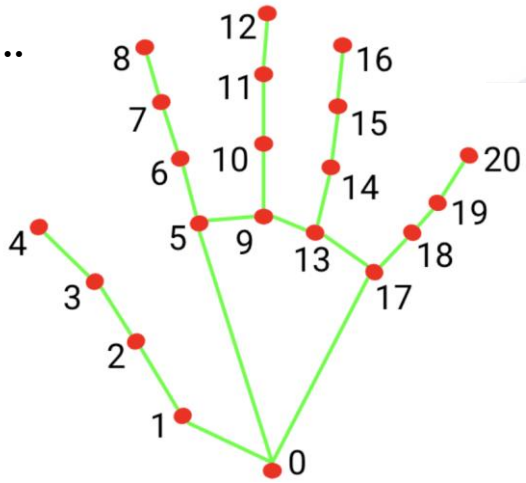
- Dobivamo stream kroz pipeline:
  1. Konfiguriraj senzore
  2. Start
  3. U beskonačnoj petlji čitaj frame
  4. Pretvori u numpy sliku
- Kod 02_realsense_feed.py

MediaPipe moduli:



1. Pose:
   - 33 ključne točke tijela
   - Skeleton: ramena, kukovi, koljena, laktovi…
2. Face Mesh:
   - 468 ključnih točaka lica
   - Detektira lice, oči, usta, obrve…
3. Hands:
   - 21 ključna točka po ruci

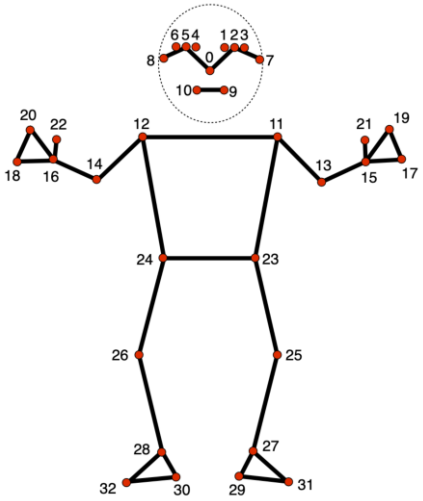


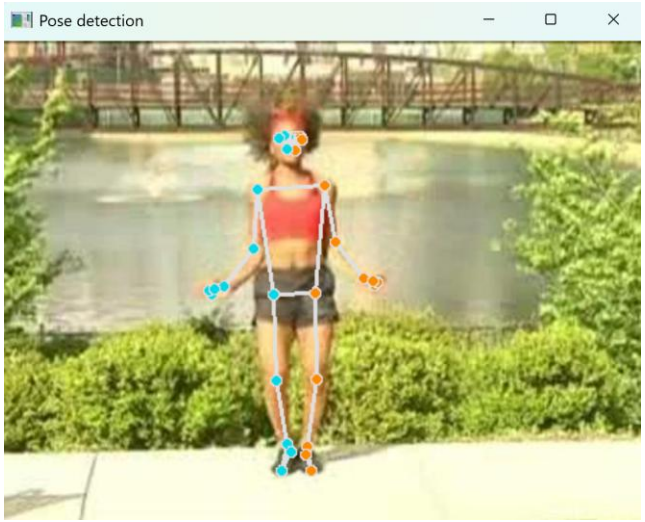


Primjer MediaPipe Pose na slici:



- OpenCV radi s BGR, a MediaPipe s RGB formatom
- Informacije o 33 ključne točke i veze između njih





- Kod 03_find_keypoints.py

Što je MediaPipe landmark?

- MediaPipe landmarks – X, Y, Z koordinate
- results.pose_landmarks.landmark - Kada primijenimo MediaPipe Pose model dobijemo listu od 33 ključne točke tijela
- Svaki landmark sadrži 4 vrijednosti:
  1. **x** – normalizirana horizontalna koordinata (0–1)
  2. **y** – normalizirana vertikalna koordinata (0–1)
  3. **z** – relativna dubina (procjena modela, ne stvarna mjera)
  4. **visibility** – vjerojatnost da je točka vidljiva (0–1)

Primjer čitanje podataka o ključnim točkama:

VRIJEDI ZA 2D SLIKE

- Normalizirane koordinate:
  - Kad MediaPipe vrati landmark.x i landmark.y, oni nisu u pikselima nego u rasponu **od 0 do 1** - Iz RGB slike NEMAMO stvarnu dubinu u metrima
  - Korisno: Slika 640×480 i slika 1920×1080 daju iste x,y vrijednosti

  - landmark.z je **procijenjena** relativna dubina modela (**ne mjerena**) treniranog na 3D podacima
  - **Dubina** - vrijednost kaže je li zglob ispred ili iza ravnine tijela

- Kod 04_get_keypoints_data.py

Primjer čitanje podataka o ključnim točkama:

VRIJEDI ZA PODATKE DOBIVENE IZ KAMERE S DUBINOM

- Vrijednosti x i y – stvarni pikseli
- Vrijednosti z – stvarna dubina [m]

- Kamera ima **intrinzične parametre** (žarišna duljina, optičko središte, geometrija senzora) – RS biblioteka nudi opciju čitanja tih podataka
- Prema poznatim informacijama očitanim u pikselima, možemo izvesti de-projekciju u 3D → vrijednosti **x i y u metrima**
- Nakon deprojekcije dobivamo (X, Y, Z) u [m] u koordinatnom sustavu kamere

- Kod 05_get_keypoints_data_RS.py

Primjer čitanje podataka o ključnim točkama:

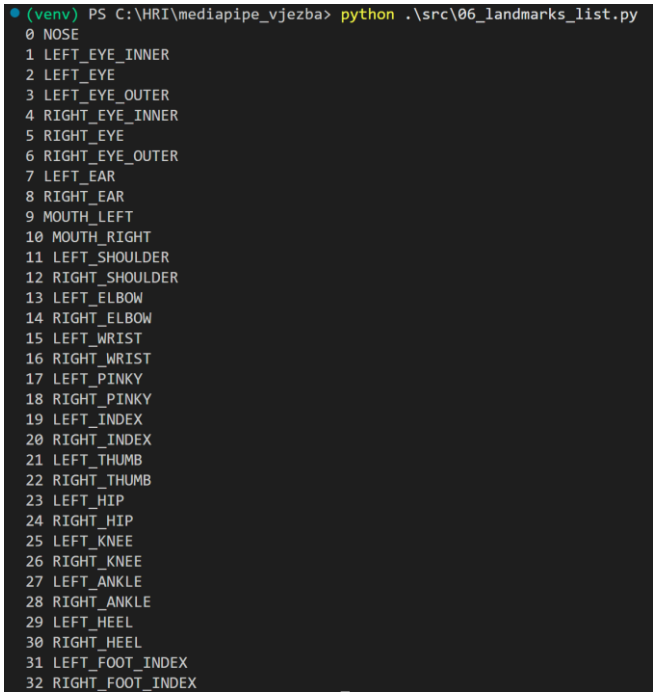
- Visibility:
  - Vrijednost u rasponu 0–1
  - Signalizira koliko je model siguran da je landmark stvarno vidljiv (pouzdanost podataka)
  - Visibility < 0.5 → točka je možda skrivena
  - Ruka iza tijela → low visibilityice okrenuto od kamere → neki landmarkovi imaju visibility = 0
  - Kod računanja kuta ili položaja moramo ignorirati landmarkove niske vidljivosti

Primjer pristupanje ključnoj točki u kodu:

- Pristupanje ključnoj točki ‘NOSE’:

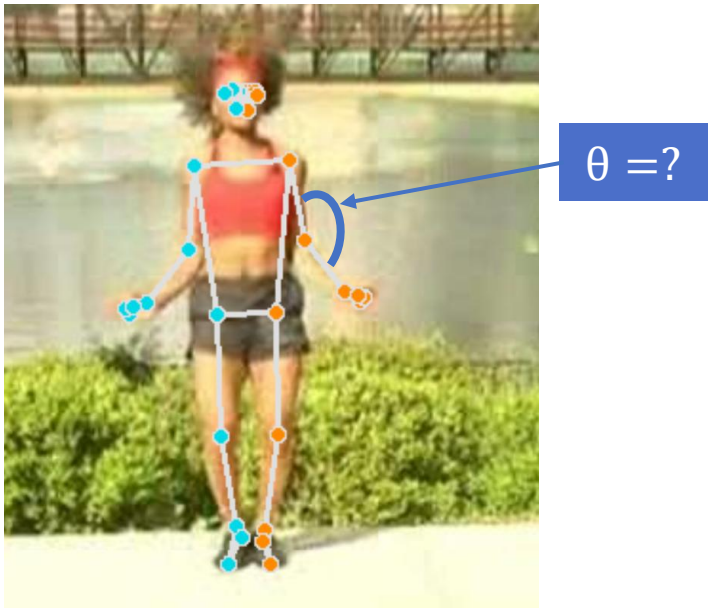
```
lm = results.pose_landmarks.landmark
nose = lm[mp_pose.PoseLandmark.NOSE.value]
```

  - Čitanje vrijednosti:
    - nose.x
    - nose.y
    - nose.z
    - nose.visibility
  - Popis svih landmarkova koje modul ima – kod 06_landmarks_list.py



Primjer izračun kuta koristeći ključne točke:

- U analizi ljudskog pokreta **kutovi između segmenata** tijela najčešće su najinformativniji parametar, jer izravno opisuju položaj i kretnju zglobova
- Za izračun kuta između dva segmenta uvijek su potrebne **tri različite ključne točke**
- **Primjer:** izračunati kut u laktu lijeve ruke

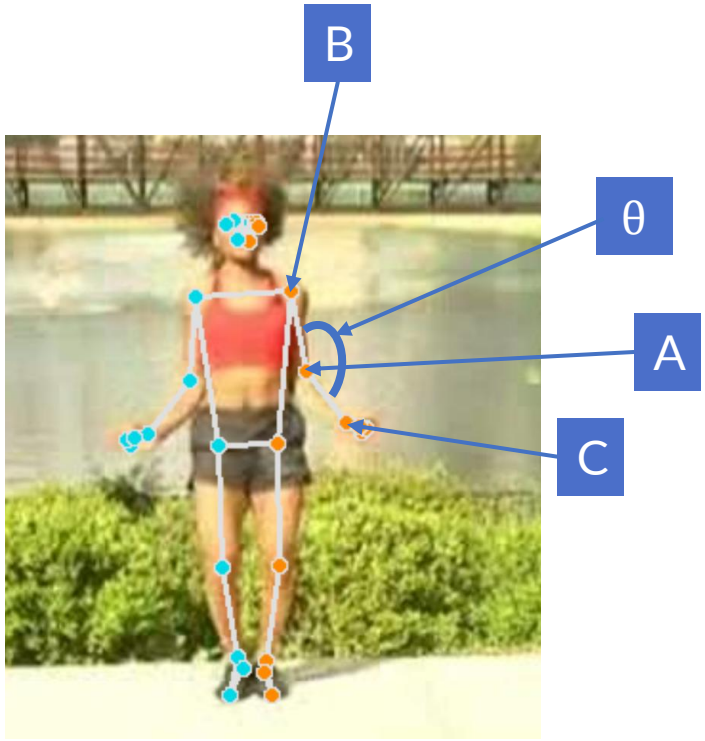


Odabrane ključne točke:

- Lijevo rame - LEFT_SHOULDER
- Lijevi lakat - LEFT_ELBOW
- Lijevo zapešće - LEFT_WRIST

Primjer izračun kuta koristeći ključne točke:

Ovo je ista formula koju koristimo u funkciji calculate_angle(a, b, c) u kodu



- Određivanje vektora:

$$\overrightarrow{AB} = B - A = (x_B - x_A, y_B - y_A)$$

$$\overrightarrow{AC} = C - A = (x_C - x_A, y_C - y_A)$$

- Matematička norma:

$$\left\|\overrightarrow{AB}\right\| = \sqrt{{x_{AB}}^2 + {y_{AB}}^2}$$

$$\left\|\overrightarrow{AC}\right\| = \sqrt{{x_{AC}}^2 + {y_{AC}}^2}$$

- Skalarni produkt:

$$\overrightarrow{AB} * \overrightarrow{AC} = x_{AB} * x_{AC} + y_{AB} * y_{AC}$$

- Kut između dva vektora:

$$\cos\theta = \frac{\overrightarrow{AB} * \overrightarrow{AC}}{\left\|\overrightarrow{AB}\right\| * \left\|\overrightarrow{AC}\right\|}$$

- Kod 07_angle_calculation.py

Javno dostupni setovi s temeljnom istinom (engl. ground truth):

- Skupovi podataka koji su dostupni za istraživače, developere i entuzijaste, besplatni te se slike i snimke smiju koristiti (GDPR)
- Omogućuju testiranje, treniranje modela i razvoj aplikacija bez potrebe za prikupljanjem vlastitih podataka
- Načini prikupljanja informacija:
  1. Optički sustavi - kamere snimaju reflektivne markere postavljene na tijelu subjekta
  2. Inercijalni senzori -nuređaji s akcelerometrima i žiroskopima za praćenje pokreta
  3. Dubinske kamere - kamere koje snimaju trodimenzionalne podatke o pokretima tijela
  4. Manualno označavanje - ručno označavanje ključnih točaka na tijelu u video zapisima

PennAction dataset:

- https://dreamdragon.github.io/PennAction/
- Sadrži 2326 video sekvenci s 15 različitih radnji i ground truth o ljudskim zglobovima za svaku sekvencu

```
/frames  ( all image sequences )
   /0001
      000001.jpg
      000002.jpg
      ...
   /0002
    ...
/labels  ( all annotations )
    0001.mat
    0002.mat
    ...
/tools   ( visualization scripts )
    visualize.m
    ...
```

Organizacija podataka

```
annotation =

      action: 'tennis_serve'
        pose: 'back'
           x: [46x13 double]
           y: [46x13 double]
  visibility: [46x13 logical]
       train: 1
        bbox: [46x4 double]
  dimensions: [272 481 46]
     nframes: 46
```

Podaci koji se mogu očitati

```
1.  head
2.  left_shoulder   3.  right_shoulder
4.  left_elbow      5.  right_elbow
6.  left_wrist      7.  right_wrist
8.  left_hip        9.  right_hip
10. left_knee       11. right_knee
12. left_ankle      13. right_ankle
```

Ključne točke o kojima postoje informacije

Primjer čitanje podataka PennAction:

- Čitamo strukturu dataseta kako bismo znali gdje se nalaze koordinate i kako su organizirane po frameovima
- Otvaramo annotation label datoteku u .mat formatu
- Ispisujemo što dataset sadrži
- Cilj nije odmah obrađivati, nego pogledati strukturu

- x → koordinate svih jointova kroz sve frameove
- y → koordinate svih jointova kroz sve frameove
- visibility → koji joint je vidljiv (1 ili 0)
- bbox → bounding box osobe
- nframes → broj frameova
- action → naziv radnje (npr. pullup)

- Kod 08_reading_data_pennaction.py

Primjer čitanje podataka PennAction – x,y:

- Cilj je izdvojiti koordinate iz dataseta za **određeni frame**
- Dobiti Python listu [x1, x2, ...] i [y1, y2, ...] - ground truth

```
1.  head
2.  left_shoulder   3.  right_shoulder
4.  left_elbow      5.  right_elbow
6.  left_wrist      7.  right_wrist
8.  left_hip        9.  right_hip
10. left_knee       11. right_knee
12. left_ankle      13. right_ankle
```

Ključne točke o kojima postoje informacije

- U PennAction-u **x i y su koordinate u pikselima** (nisu normalizirane)

- Kod 09_reading_data_pennaction_xy.py

Samostalni zadatak 1:

Usporedba ground truth i MediaPipe

- Odabrati 5 fotografija iz mape podaci
- Za svaku fotografiju učitati ground truth iz PennAction dataseta
- Odabrati proizvoljnu ključnu točku koju želite usporediti
- Za istu ključnu točku dobiti poziciju pomoću MediaPipe Pose
- Rezultate usporediti i prikazati u tablici

ID slike	Ground truth (x, y)	MediaPipe (x, y)	Razlika

Samostalni zadatak 2:

Izračun kuta zgloba

- Odabrati 5 fotografija iz mape podaci
- Izračunati kut:
  - desni lakat
  - lijevo koljeno
- Ispisati rezultate u terminal i prikazati tablicu

ID slike	Lakat (°)	Koljeno (°)

Samostalni zadatak 3:

Nagib tijela

- Odabrati 5 fotografija iz mape podaci
- Sami odabrati ključne točke torza
- Izračunati nagib u odnosu na vertikalu
- Rezultat prikazati u tablici

ID slike	Nagib (°)

Samostalni zadatak 4:

Live tracking i brzina ruke

- Koristiti live feed iz kamere
- Pratiti poziciju zapešća u vremenu
- Izračunati brzinu gibanja u m/s
- Spremiti rezultate u .txt datoteku

Vježba 2

Head pose estimation

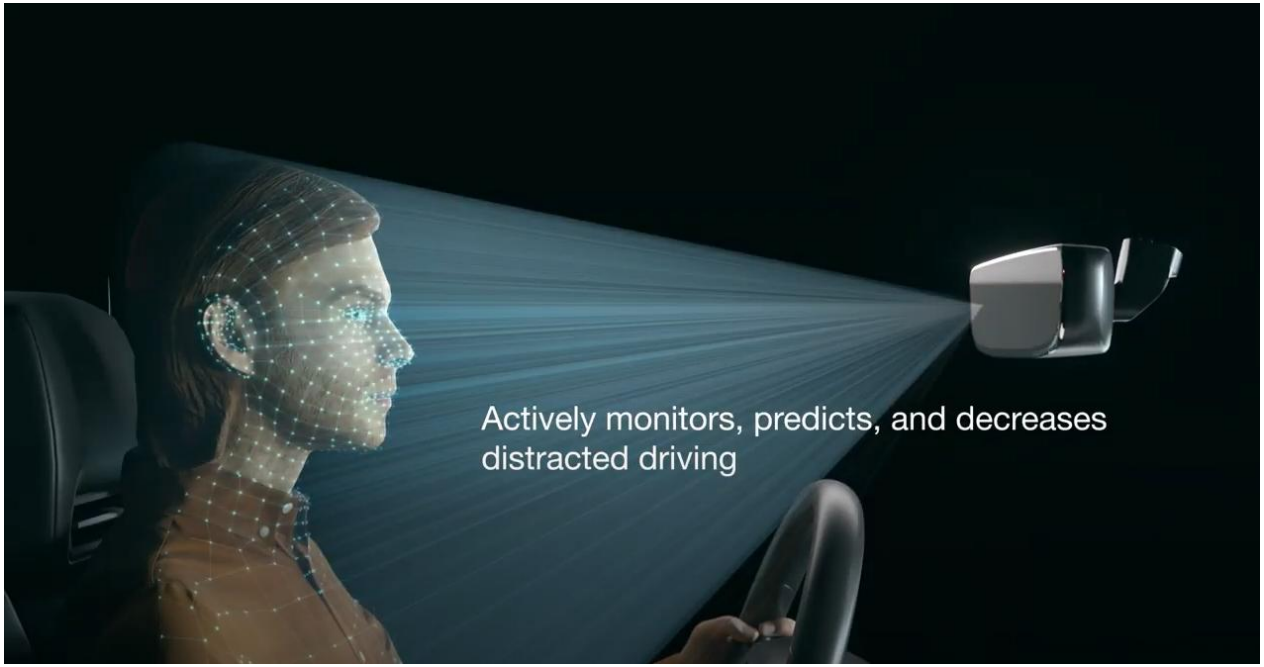
Zašto nam je HPE korisan u HRI?

- Poveznica između interakcije čovjeka i robota te vizijskih sustava je direktna: kamere su jedan od ključnih senzora za **percepciju okoline**

- Primjene:
  - robot može prepoznati smjer pogleda korisnika
  - sigurnija kolaboracija u fizičkoj interakciji (pHRI)
  - socijalna interakcija: robot zna kad ga gledamo
  - humanoidni roboti i asistivni sustavi
  - praćenje pažnje i detekcija umora

Gdje se još koristi HPE?

- self-driving i monitoring vozača (detekcija umora/pogleda)



Gdje se još koristi HPE?

- self-driving i monitoring vozača (detekcija umora/pogleda)
- AR/VR – praćenje pogleda i korisničkog fokusa



Gdje se još koristi HPE?

- self-driving i monitoring vozača (detekcija umora/pogleda)
- AR/VR – praćenje pogleda i korisničkog fokusa
- medicinska robotika i asistivna tehnologija

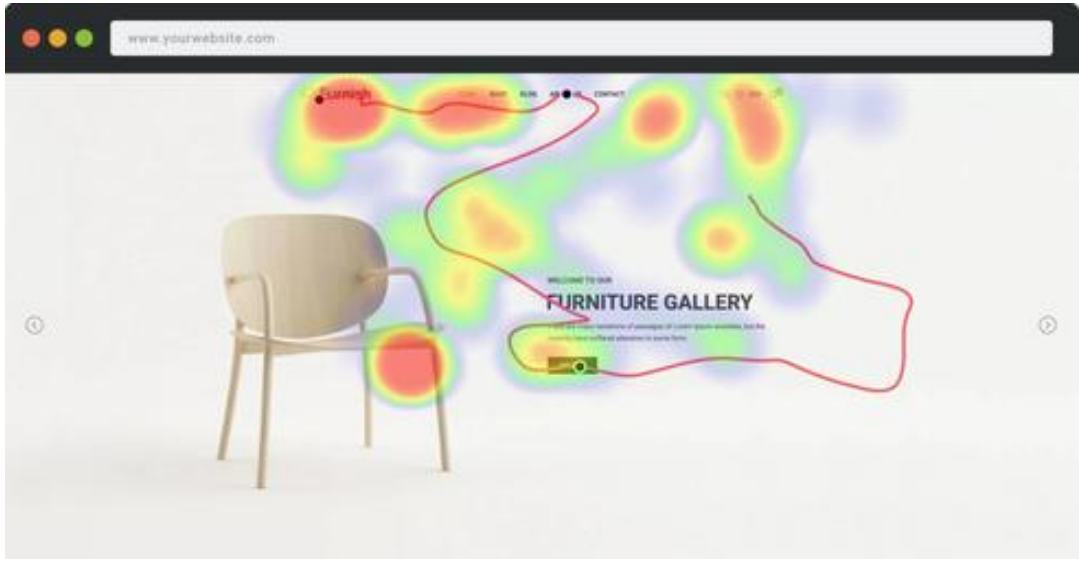
Gdje se još koristi HPE?

- self-driving i monitoring vozača (detekcija umora/pogleda)
- AR/VR – praćenje pogleda i korisničkog fokusa
- medicinska robotika i asistivna tehnologija
- sigurnosni sustavi i videoanalitika



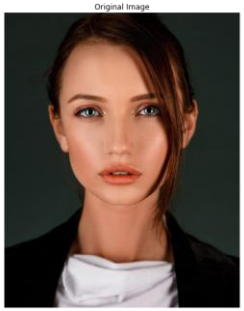
Gdje se još koristi HPE?

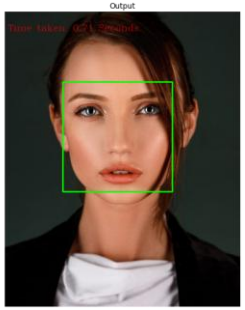
- self-driving i monitoring vozača (detekcija umora/pogleda)
- AR/VR – praćenje pogleda i korisničkog fokusa
- medicinska robotika i asistivna tehnologija
- sigurnosni sustavi i videoanalitika
- UI/UX gdje se interfejs mijenja prema smjeru pogleda

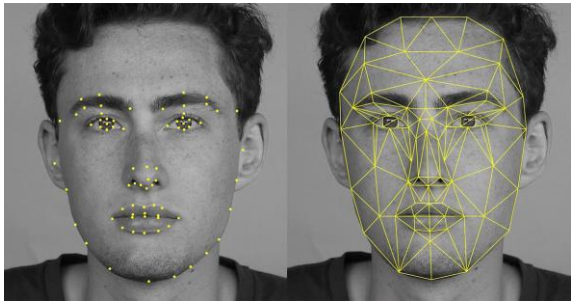


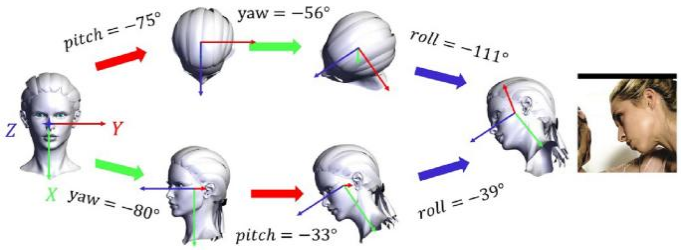
Kako se procjenjuje orijentacija glave?

- Opći pristup u HPE metodama:
  1. Detekcija lica
  2. Ekstrakcija značajki/ključnih točaka
  3. Modeliranje geometrije (2D ili 3D)
  4. Izračun rotacije glave





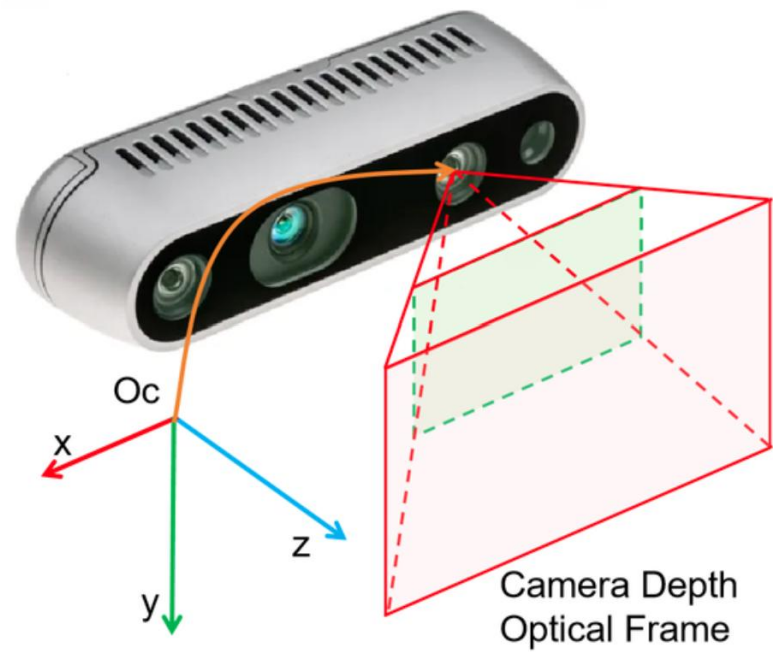




Referentni koordinatni sustav (Camera Space):

- Kad računamo head pose, uvijek mjerimo orijentaciju u odnosu na kameru
- **Head pose** = rotacija u odnosu na kameru, ne u odnosu na tijelo

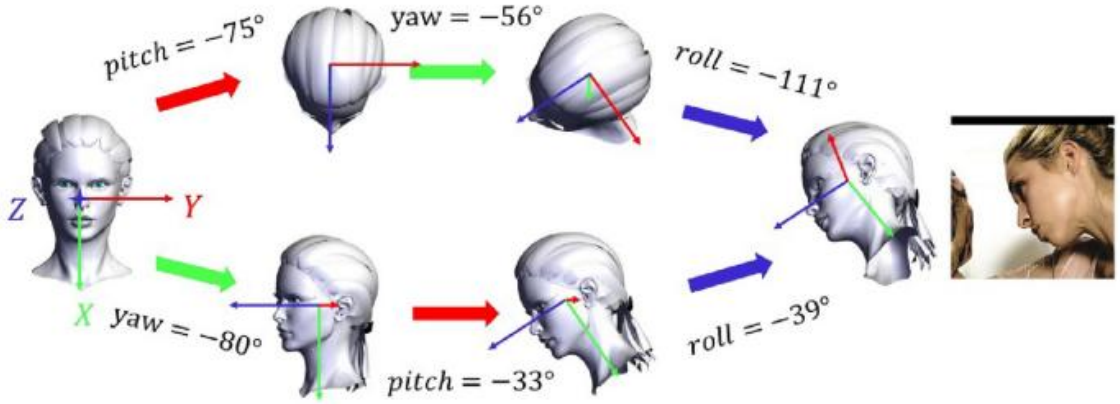
- Camera-space (MediaPipe/OpenCV):
  - kamera definira ishodište koordinata
  - x – horizontalno
  - y – vertikalno
  - z – dubina



Eulerove rotacije:

= Način opisivanja orijentacije tijela u prostoru pomoću tri uzastopne rotacije oko osi koordinatnog sustava

- Svaka rotacija mijenja smjer sljedeće osi → **redoslijed rotacija je vrlo važan**

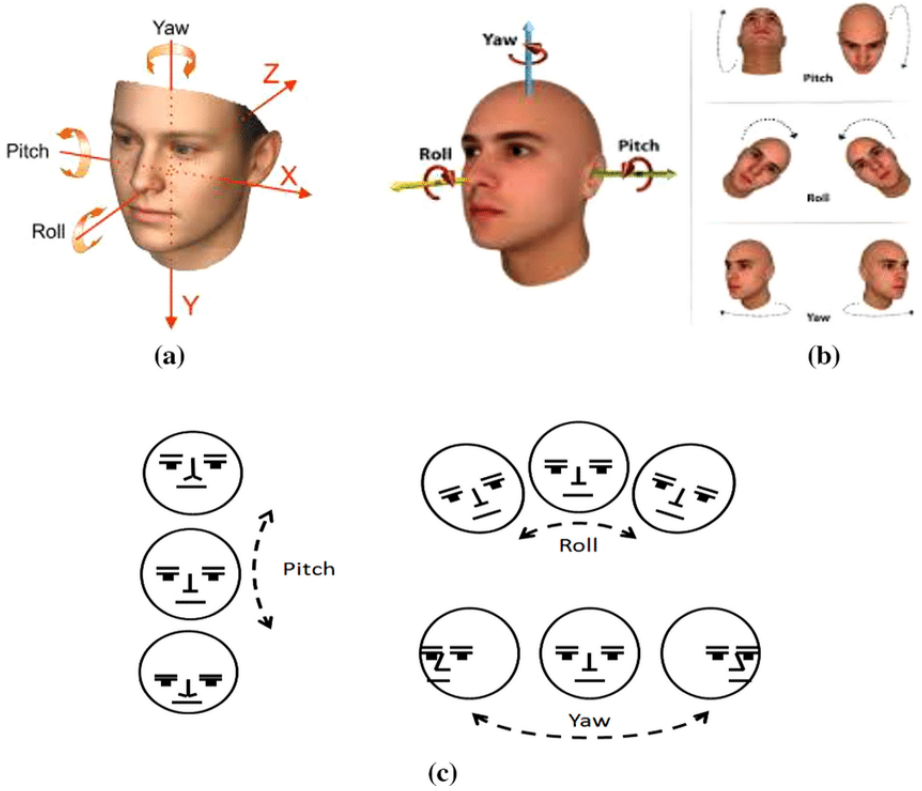


- Najčešće korišten sustav u računalnom vidu: Pitch – Yaw – Roll (rotacije oko X, Y, Z osi)

Computer Vision Head Pose Convention:

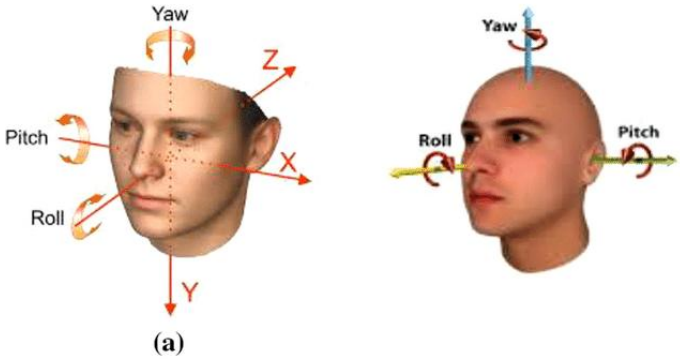
- Kutovi ovise o Camera Space referentnom koordinatnom sustavu
- Postoje i druge konvencije (ovisno o području primjene: robotika, avijacija, računalni vid, VR), mi ćemo **za HPE koristiti sljedeću definiciju**:

Kut	Os	Značenje
Pitch	oko X	glava gore/dolje
Yaw	oko Y	glava lijevo/desno
Roll	oko Z	nagib glave



Smjer z-osi:

- U literaturi za Head Pose Estimation ne postoji jedna jedina univerzalna konvencija
- Dva jednako ispravna, ali različita pristupa:



1. **Z-os ide prema nosu** (ulazi u glavu):
    - Koristi se u: računalnom vidu (OpenCV standard camera coordinate system), mnogim HPE radovima, BIWI Head Pose datasetu
2. **Z-os ide iz glave prema kameri** (izlazi iz nosa):
    - Koristi se u: računalnoj grafici, 3D animaciji, AR/VR sustavima

- Obje definicije su ispravne, ono što je bitno je uvijek znati oji koordinatni sustav koristi dataset ili algoritam

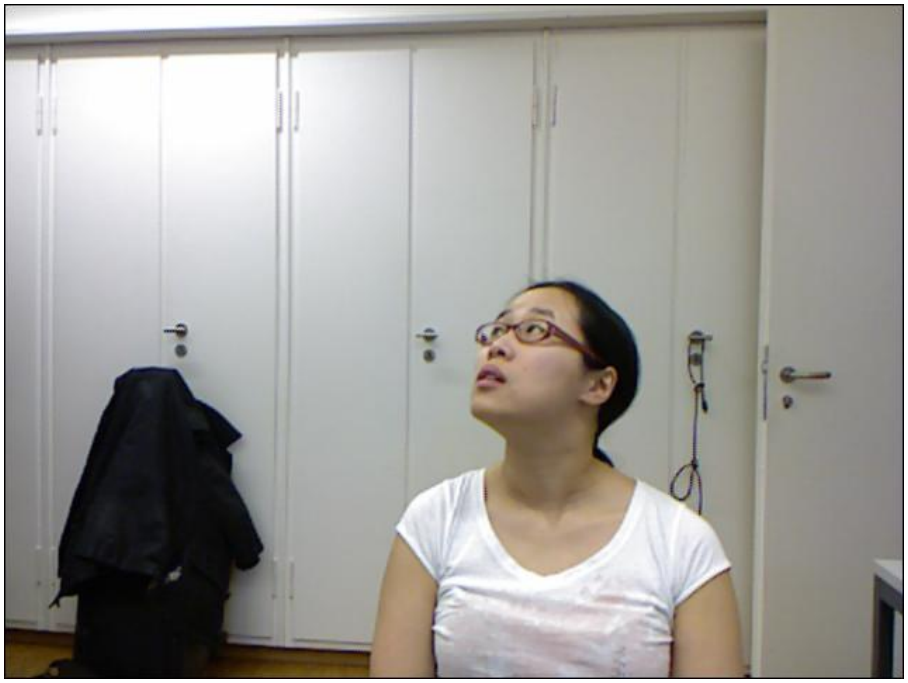
BIWI dataset:

- https://www.kaggle.com/datasets/kmader/biwi-kinect-head-pose-database
- Jedan od najkorištenijih datasetova za HPE u računalnom vidu
- Institucija: ETH Zürich (Computer Vision Lab)
- Senzor: Microsoft Kinect (RGB + Depth)
- Broj sudionika: 24
- Ukupno: ~15,000 RGB-D slika
- Daje ground truth rotacije (matrica rotacija) → većina datasetova daje samo 2D ključne točke
- Ima dubinu → moguće je točno rekonstruirati 3D glavu i orijentaciju

Napomena: pravi dataset je skinut sa službenih stranica, pa koristimo verziju postavljenu na Kaggle (neki podaci nedostaju)

Primjer - Što BIWI daje za svaki frame:

1. RGB slika (rgb.png)
    - 640×480 format, slika u boji
    - Koristi se za Head Pose Estimation model (npr. MediaPipe, YOLO, 3D landmark modeli…)

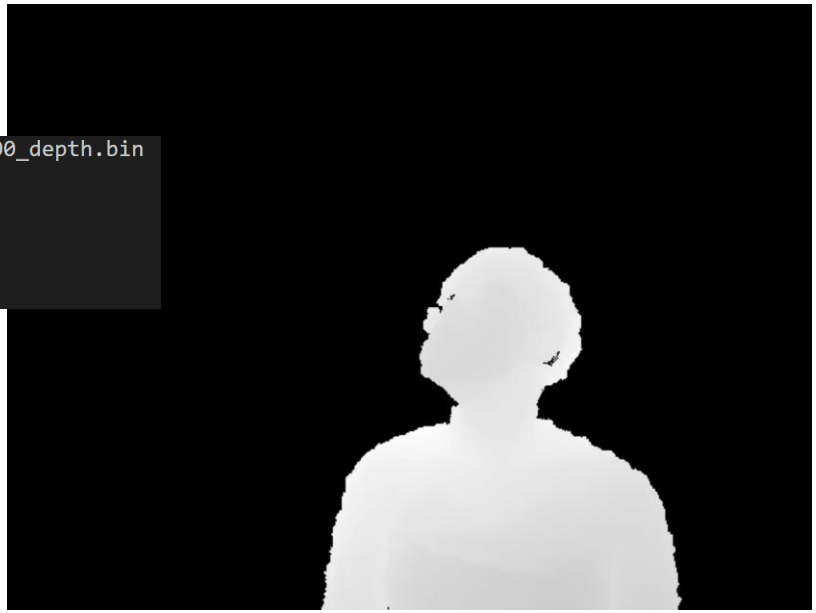


Folder 03, frame_00100_rgb.png

Primjer - Što BIWI daje za svaki frame:

2. Depth mapa (depth.bin)
   - Komprimirani zapis dubinske slike → slika se sastoji od 640×480 float vrijednosti (u milimetrima!)
   - Predstavlja stvarnu udaljenost svakog piksela od kamere
   - Potrebna je posebna funkcija za dekodiranje (decode_biwi_depth)
   - Može se vizualizirati kao grayscale depth slika

```
Čitam depth iz: C:\HRI\mediapipe_vjezba\data\raw\hpe\03\frame_00100_depth.bin
Dimenzije depth mape: 640 x 480
Tip podataka: float32
Min dubina (m): 0.797
Maks dubina (m): 0.997
Depth u centru (px=(320,240)) = 0.000 m
```



Kod 01_depth_info.py (frame_00100_depth.bin)

Primjer čitanje i vizualizacija podataka *depth.bin*:

- Funkcija ‘decode_biwi_depth’ – Čita raw .bin datoteku iz BIWI-a
  - Korak 1 — čitanje headera: U prvih 8 bajtova zapisani su width i height
  - Korak 2 — dekodiranje dubine: BIWI depth format koristi kompresiju “skip & read”, svaka vrijednost dubine je 2 bajta (uint16)
  - Korak 3 — rekonstrukcija depth slike: Depth se reshape-a u matricu (height × width)
  - Napomena: BIWI ima 20 px border → dodaje se rub pa resize na 640×480

Kod 01_depth_info.py (frame_00100_depth.bin)

Primjer čitanje i vizualizacija podataka *depth.bin*:

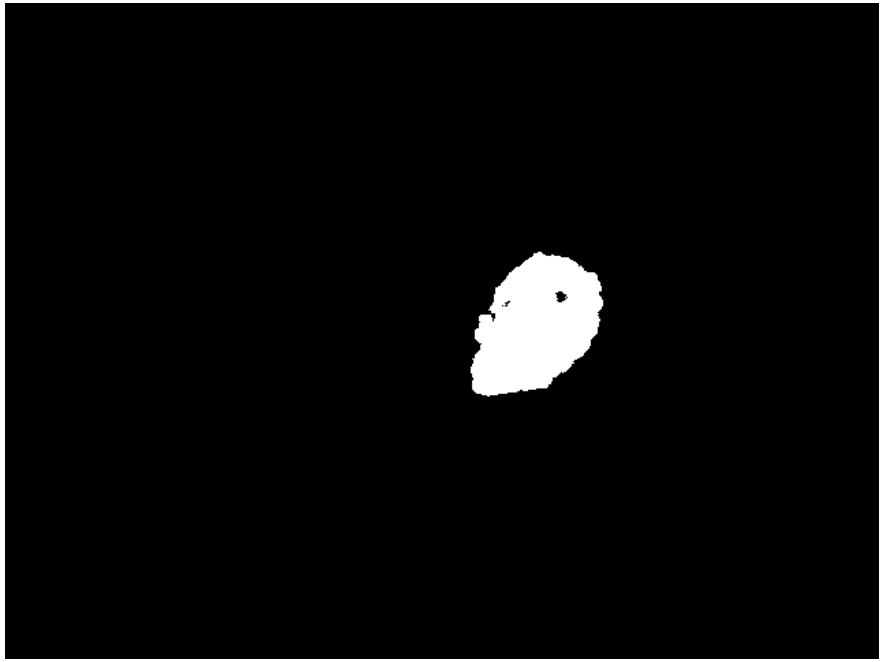
- Analiza dubinske mape (ispis podataka):
  - Minimalna dubina: najbliža točka glavi
  - Maksimalna dubina: najudaljenija točka glavi
  - Depth u centru slike (px = (320,240))
- Analiza dubinske mape (vizualizacija):
  - Depth normaliziramo na raspon 0–255
  - Pretvaramo u grayscale sliku koju OpenCV može prikazati
  - Svjetliji piksel → veća dubina (ili obrnuto, ovisno o normalizaciji)



Kod 01_depth_info.py (frame_00100_depth.bin)

Primjer - Što BIWI daje za svaki frame:

3. Binary mask (mask.png)
    - 0 = pozadina
    - 255 = silueta glave
    - Pomaže kod segmentacije / traininga mreža koje trebaju samo glavu.



Kod 01_depth_info.py (frame_00100_depth.bin)

Primjer - Što BIWI daje za svaki frame:

4. Rotaciju i translaciju u odnosu na koordinatni sustav kamere (pose.txt)
    - Napomena: pose.txt i pose.bin sadrže iste informacije, razlika je u vrsti zapisa podataka
    - Struktura pose.txt datoteke:
        - Datoteka ima 4 retka × 3 stupca:

            prva 3 retka → rotacijska matrica R (3×3)

            zadnji red → translacija T (tx, ty, tz) u milimetrima

Kod 02_pos_rot_info.py (frame_00100_pose.txt)

Primjer - Što BIWI daje za svaki frame:

- Funkcija ‘load_biwi_pose_txt’ – Čita raw .txt datoteku iz BIWI-a
  - Učitava matricu iz .txt
  - Odvaja R i T
  - T automatski pretvara u metre (dijeljenje s 1000)
  - Vraća:
    - R → rotacijska matrica
    - T_m → vektor pozicije glave u metrima

Kod 02_pos_rot_info.py (frame_00100_pose.txt)

Primjer - Što BIWI daje za svaki frame:

- Funkcija ‘rotmat_to_euler_xyz’ – pretvara rotacijsku matricu R u Eulerove kutove (pitch, yaw, roll), mjerna jedinica su stupnjevi
  - Prema dogovoru:
    - pitch → rotacija oko X (gore–dolje)
    - yaw → rotacija oko Y (lijevo–desno)
    - roll → rotacija oko Z (nagib glave)
    - koristi redoslijed rotacija XYZ (kompatibilno s BIWI postavkama)

$$R = \begin{bmatrix} R_{00} & R_{01} & R_{02} \\ R_{10} & R_{11} & R_{12} \\ R_{20} & R_{21} & R_{22} \end{bmatrix} \xrightarrow{\text{konverzija}}$$

$$pitch = \arctan 2(R_{21}, R_{22})$$

$$yaw = \arctan 2\left(-R_{20}, \sqrt{{R_{00}}^2 + {R_{10}}^2}\right)$$

$$roll = \arctan 2(R_{10}, R_{00})$$

Kod 02_pos_rot_info.py (frame_00100_pose.txt)

Primjer - Što BIWI daje za svaki frame:

- Analiza podataka o poziciji i rotaciji (ispis podataka):
  - matricu R
  - poziciju glave T (m)
  - Euler kutove:
  - Pitch (oko X)
  - Yaw (oko Y)
  - Roll (oko Z)

```
Rotacijska matrica R:
[[ 0.730452  -0.373211   0.571973 ]
 [-0.0121179  0.830273   0.557226 ]
 [-0.682857  -0.413958   0.601952 ]]
[[ 0.730452  -0.373211   0.571973 ]
 [-0.0121179  0.830273   0.557226 ]
 [-0.682857  -0.413958   0.601952 ]]

Translacija T (m):
tx = 0.090 m, ty = 0.004 m, tz = 0.904 m

Euler kutovi (xyz):
Pitch (oko X) = -34.52°
Yaw   (oko Y) = 43.07°
Roll  (oko Z) = -0.95°
```

Kod 02_pos_rot_info.py (frame_00100_pose.txt)

Od rotacijske matrice do Eulerovih kutova:

- Matrica rotacije je 3D transformacija koja opisuje kako je glava orijentirana u odnosu na kameru

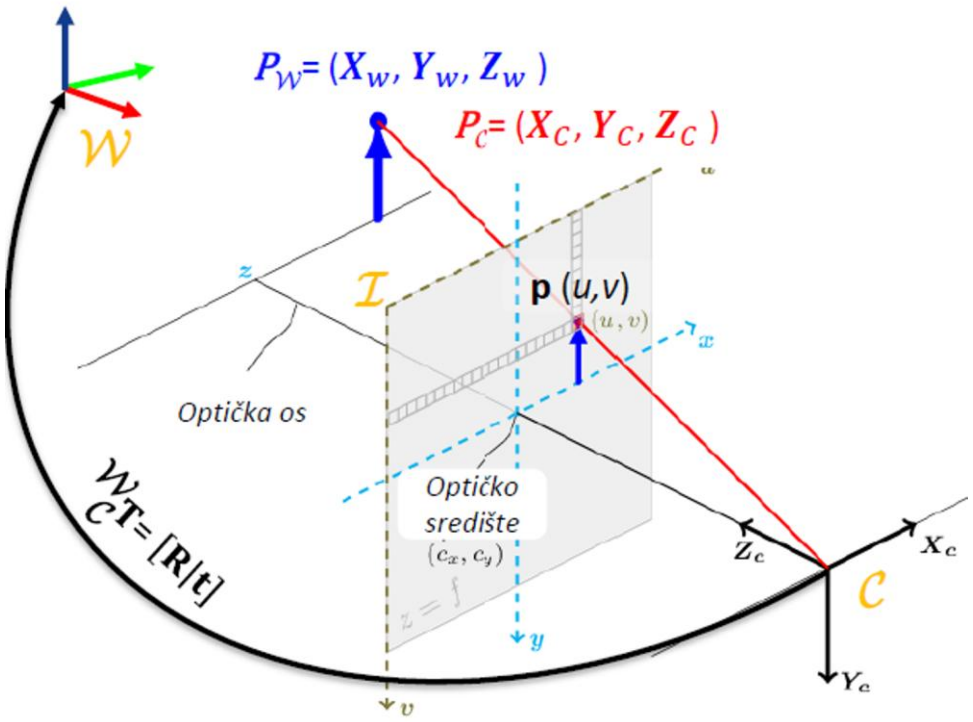
$$R = \begin{bmatrix} R_{00} & R_{01} & R_{02} \\ R_{10} & R_{11} & R_{12} \\ R_{20} & R_{21} & R_{22} \end{bmatrix}$$

- Stupci definiraju smjer osi (x, y, z)
- Ishodište koordinatnog sustava definirano je pozicijom centra glave
- Duljine osi proizvoljno su postavljene na 200 mm
- Definiramo točku ishodišta te završne točke x, y i z osi
- Problem – vrijednosti su u mm, da ih vizualiziramo trebamo ih u pikselima

Kod 03_vis_pos_rot_info.py (frame_00100_pose.txt, frame_00100_rgb.png)

Od rotacijske matrice do Eulerovih kutova:

- Projekcija 3D točke u 2D piksele pomoću K (intrinzične matrice kamere)



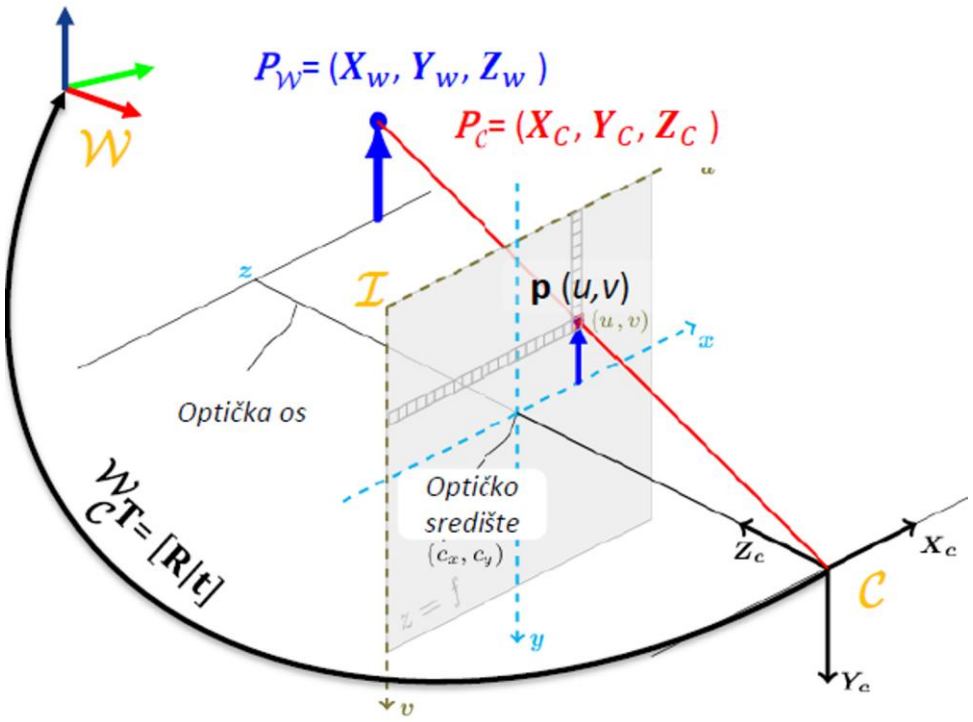
$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_C \\ Y_C \\ Z_C \end{bmatrix}$$

- Intrinzična matrica kamere pretvara 3D koordinate X_C, Y_C, Z_C u 2D piksele u, v
- Pinhole model kamere

f_x, f_y = žarišna udaljenost [mm]
c_x, c_y = optičko središte

Od rotacijske matrice do Eulerovih kutova:

- Projekcija 3D točke u 2D piksele pomoću K (intrinzične matrice kamere)



$$p = K \cdot P$$

$$u = \frac{px}{pz}$$

$$v = \frac{py}{pz}$$

- Pomoću ovih odnosa dobijemo točke ishodišta i krajeva osi u pikselima

Kod 03_vis_pos_rot_info.py (frame_00100_pose.txt, frame_00100_rgb.png)

Primjer - Što BIWI daje za svaki frame:

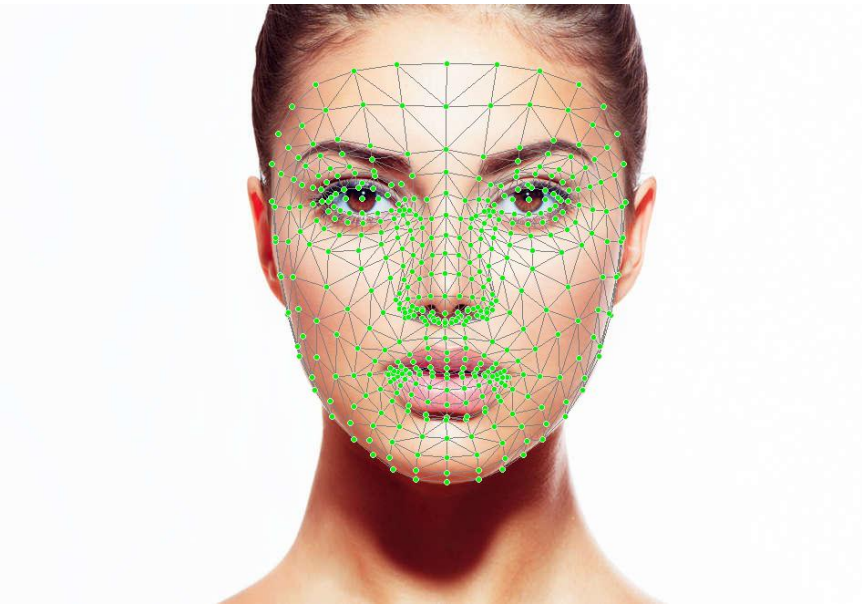
- Analiza podataka o poziciji i rotaciji (vizualizacija):



Kod 03_vis_pos_rot_info.py (frame_00100_pose.txt, frame_00100_rgb.png)

Primjer – MediaPipe FaceMesh:

- Inicijalizacija FaceMesh modela - mp_face_mesh = mp.solutions.face_mesh
- Model vraća 468 3D landmarka za svako detektirano lice





Kod 04_face_mesh.py (frame_00100_rgb.png)

Podjela dataseta:

- Nikola Akrap → 05
- Antonio Ćuk → 10
- Bruno Gugo → 17
- Krešimir Hartl → 24
- Ivan Noršić → 11
- Matija Pongračić → 21

Samostalni zadatak 1:

Vizualna interpretacija BIWI Head Pose podataka

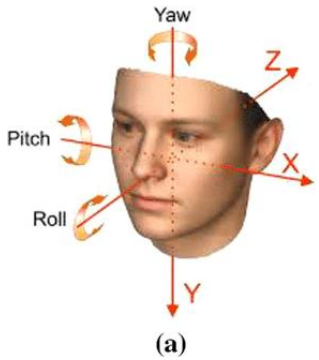
- Za 3 RGB slike iz BIWI dataseta:
  - Učitati RGB sliku, te informacije o rotaciji i translaciji
  - Izračunati Eulerove kutove (XYZ konvencija)
  - Nacrtati osi glave na slici
  - Opišite kamo čovjek gleda (gleda li osoba gore/dolje?, gleda li lijevo/desno?, naginje li glavu?, je li BIWI pogodio ili izgleda krivo?)
  - Rezultate usporediti i prikazati u tablici te tablicu priložiti u izvješće

Frame	Pitch	Yaw	Roll	Opis

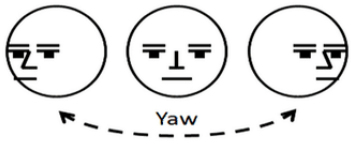
Samostalni zadatak 2:

Klasifikacija smjera pogleda

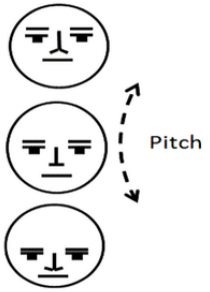
- Odabrati 3 fotografija iz mape i podatke temeljiti na ground truth vrijednostima
- Implementirati jednostavnu klasifikaciju smjera glave



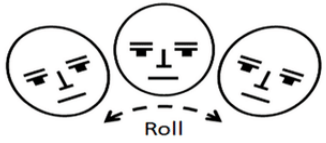
Iznos	Zaključak
$yaw < -10$	Osoba gleda ulijevo
$-10 \leq yaw < 10$	Osoba gleda naprijed
$yaw > 10$	Osoba gleda udesno



Iznos	Zaključak
$pitch < -10$	Osoba gleda dolje
$-10 \leq pitch < 10$	Osoba gleda neutralno
$pitch > 10$	Osoba gleda gore



Iznos	Zaključak
$roll < -10$	Nagib glave lijevo
$-10 \leq roll < 10$	Nema nagiba glave
$roll > 10$	Nagib glave desno



Samostalni zadatak 2:

Klasifikacija smjera pogleda

- Rezultate usporediti i prikazati u tablici te tablicu priložiti u izvješće

Frame	Pitch	Yaw	Roll	Klasifikacija pogleda (sva tri kuta)

Samostalni zadatak 3:

MediaPipe vs. BIWI: Usporedba orijentacije glave

- Za 3 RGB slika iz BIWI dataseta:
  - Pokrenuti MediaPipe Face Mesh
  - Definirati 3 ključne točke (lijevo oko, desno oko i nos)
  - Izračunati vektore iz navedenih točaka:
    - x-os = vektor između očiju
    - Z-os
    - Y-os
- Očitati matricu rotacija ground truth podataka te izračunati Eulerove kutove
- Rezultate usporediti i prikazati u tablici te tablicu priložiti u izvješće

Frame	Pitch MediaPipe	Pitch BIWI	Yaw MediaPipe	Yaw BIWI	Roll MediaPipe	Roll BIWI

Vježba 3

Gesture recognition

Gesture Recognition:

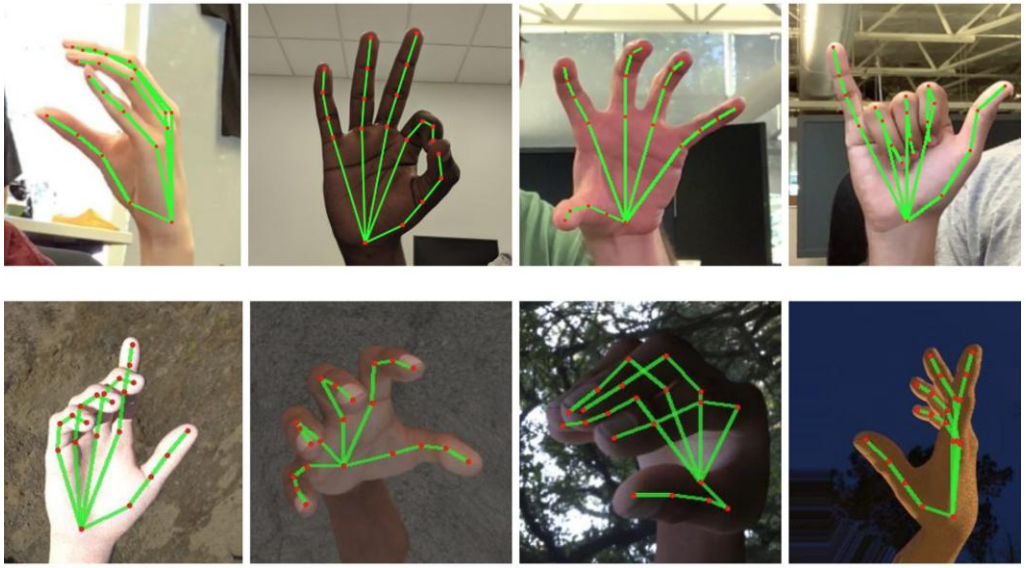
- Interakcija čovjek–računalo (robot) putem gesti ruku
- Primjena računalnog vida i strojnog učenja
- Python + MediaPipe kao alat za praktične demonstracije

- Prirodniji oblik komunikacije od tipkovnice/miša
- Hands-free interakcija u raznim okruženjima
- Ključna tehnologija u AR/VR sustavima
- Korisno u robotici, gaming industriji, pametnim uređajima
- Povećava pristupačnost (assistive technology)

Kako funkcionira gesture recognition?

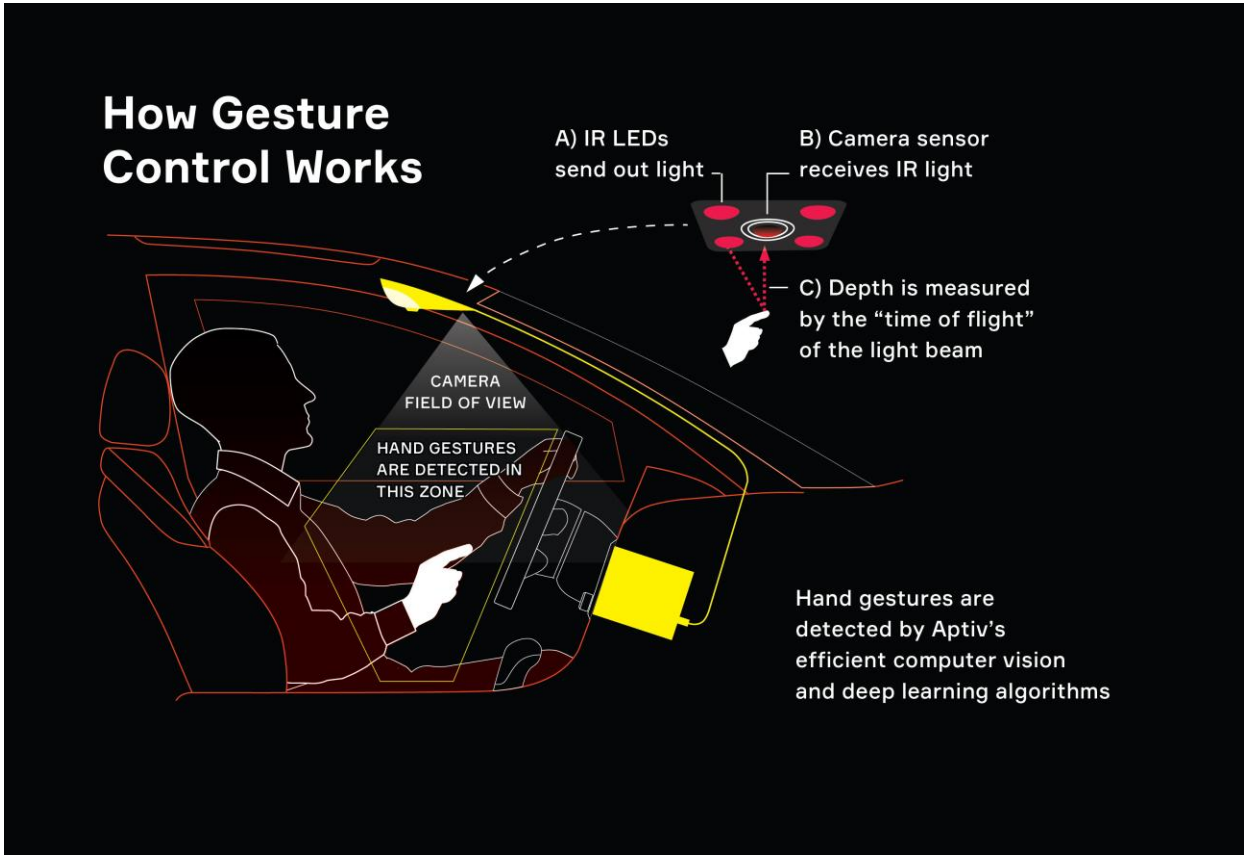
- Video/slika → detekcija ruke → landmarks → klasifikacija prema medusobnom odnosu ključnih točaka
- Izdvajanje 2D/3D točaka na ruci (21 ključna točka - MediaPipe Hands)
- Analiza položaja prstiju i njihovih međusobnih odnosa
- Korištenje ML modela za prepoznavanje uzoraka





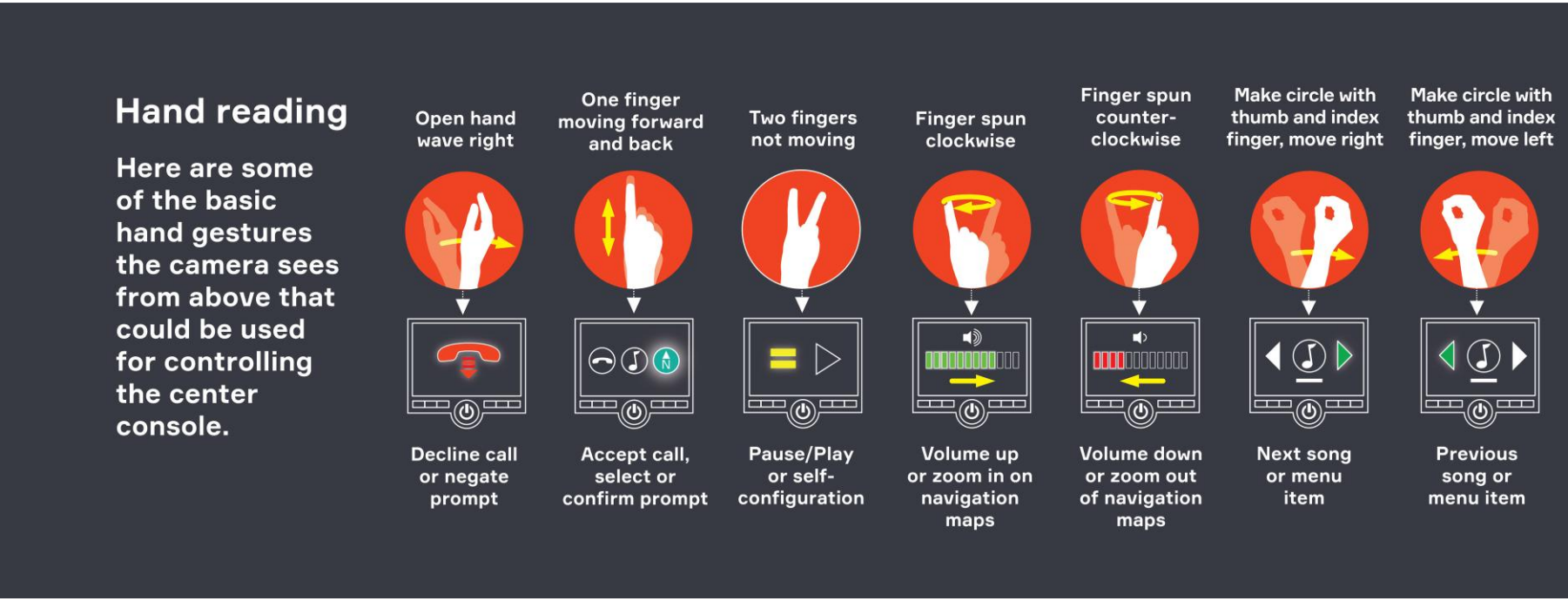
Primjeri iz svakodnevnog života:

- Automobilska sučelja bez dodira (prepoznavanje pokreta rukom za javljanje poziva ili promjenu radija)



Primjeri iz svakodnevnog života:

- Automobilska sučelja bez dodira (prepoznavanje pokreta rukom za javljanje poziva ili promjenu radija)



Primjeri iz svakodnevnog života:

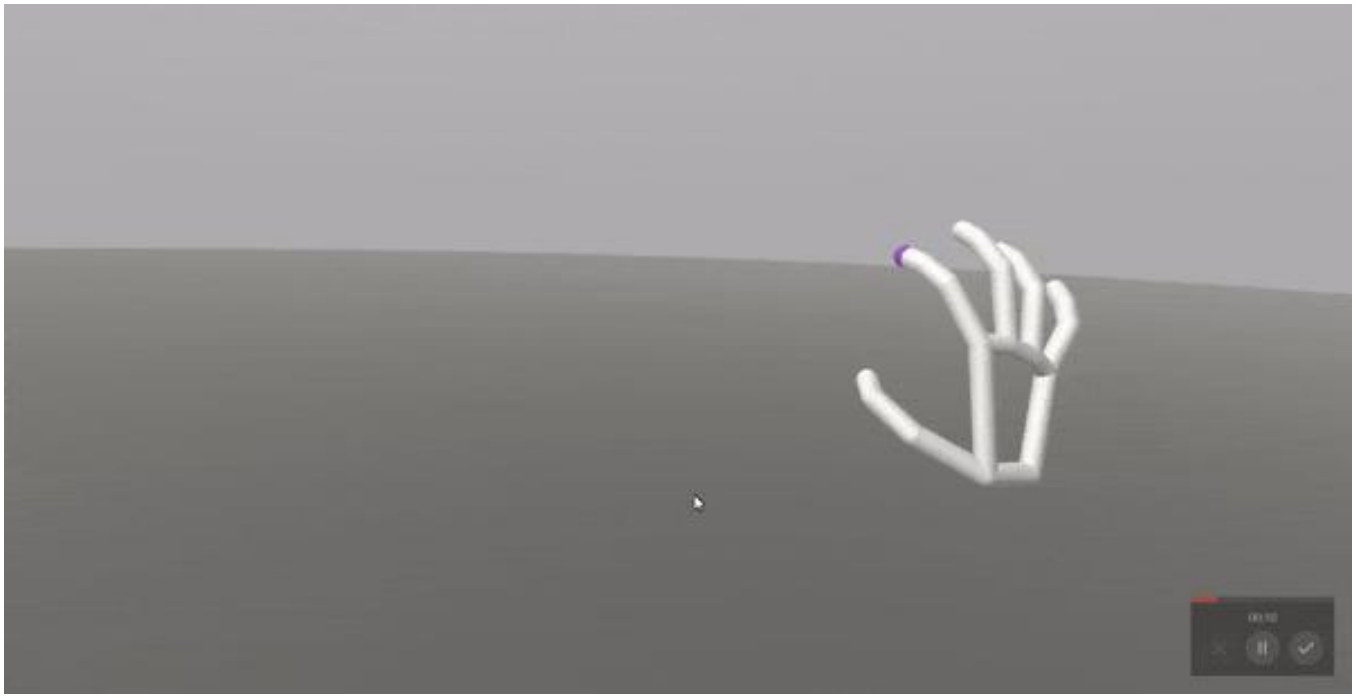
- Igranje igara gestama (“motion gaming”)
- Microsoft Kinect (Xbox 360 / Xbox One) - Najpoznatiji sustav motion gaminga bez kontrolera
- Primjer: Kinect Adventures





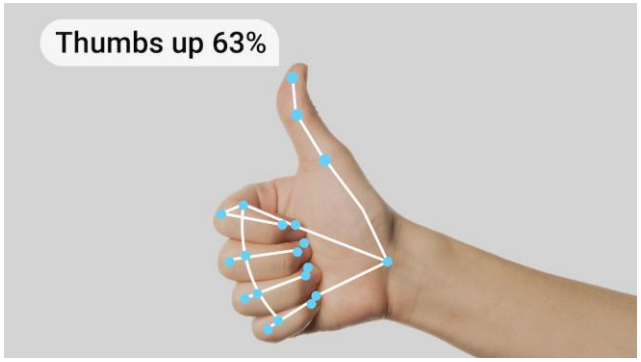
Primjeri iz svakodnevnog života:

- Interakcija u virtualnoj stvarnosti (hvatanje, povlačenje, manipulacija objektima rukom)
- Primjer: pinch kao naredba za početak crtanja



MediaPipe Gesture Recognizer:

- Gotov modul za klasifikaciju gesti
- Kombinira:
    1. Detekciju ruke
    2. Tracking
    3. Interpretaciju gesti



- Daje direktno label: "Open_Palm", "Closed_Fist", "Pointing_Up" itd.
- Jednostavan Python API

MediaPipe Gesture Recognizer:

- Kako bi ovaj modul radio, potrebno je skinuti „gesture_recognizer.task”
- .task datoteka unutar sebe sadrži unaprijed istreniran ML model za prepoznavanje gesti ruke
- Omogućuje da prepoznavanje radi real-time bez treniranja vlastitog klasifikatora
- MediaPipe Gesture Recognizer ne radi bez modela - mi mu samo šaljemo RGB sliku — model odlučuje koju gestu vidi

Primjeri gesti koje ćemo prepoznati:

- Geste klasificirane u „gesture_recognizer.task”

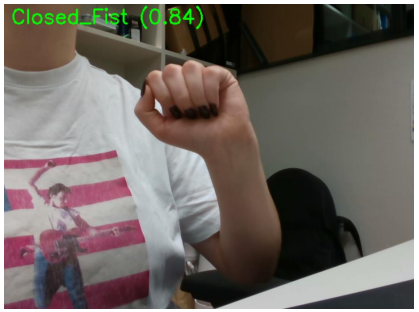
```
0 - Unrecognized gesture, label: Unknown
1 - Closed fist, label: Closed_Fist
2 - Open palm, label: Open_Palm
3 - Pointing up, label: Pointing_Up
4 - Thumbs down, label: Thumb_Down
5 - Thumbs up, label: Thumb_Up
6 - Victory, label: Victory
7 - Love, label: ILoveYou
```

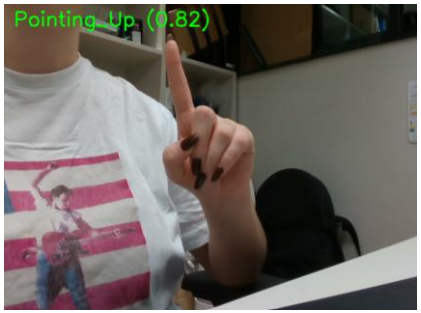
Primjer – implementacija Motion Gesture Recognition demo:

- Demo:
  - U realnom vremenu čita video stream s RealSense RGB kamere
  - Svaki frame pretvara u RGB format (potreban za MediaPipe)
  - MediaPipe **Gesture Recognizer** model (gesture_recognizer.task):
    - detektira ruku
    - analizira landmarks
    - klasificira gestu (thumbs_up, fist, open_palm…)
  - Rezultat (naziv geste + score) se ispisuje direktno na video









Kod 01_demo.py (RealSense camera feed)

Samostalni zadatak 1:

Gesture Recognition demo

- Potrebno je pokrenuti gotovi Gesture Recognition demo s RealSense kamerom
- Testirati sljedeće geste ispred kamere:
  - 👍 → thumbs up
  - 👎 → thumbs down
  - ☝ → point up
  - ✊ → closed fist
  - 🖐 → open palm
- Napraviti screenshot svake ispravno prepoznate geste

Samostalni zadatak 2:

Ručno definirana pravila za nove geste

- Izabrati dvije nove geste koje nisu dio zadanog MediaPipe modela, npr.:
  - ✌️ → peace
  - 👉 → pointing right
  - 🤘 → rock on
  - 🤙 → call me
- Koristeći MediaPipe Hands landmarke, definirati jednostavna logička pravila, npr.:
  - uspoređivanje y-koordinata vrhova prstiju
  - provjera jesu li neki prsti savijeni/ispruženi
  - udaljenosti između prstiju
  - kutovi između segmenata prstiju

Samostalni zadatak 2:

Ručno definirana pravila za nove geste

- Izabrati dvije nove geste koje nisu dio zadanog MediaPipe modela, npr.:
  - ✌️ → peace
  - 👉 → pointing right
  - 🤘 → rock on
  - 🤙 → call me
- Koristeći MediaPipe Hands landmarke, definirati jednostavna logička pravila, npr.:
  - uspoređivanje y-koordinata vrhova prstiju
  - provjera jesu li neki prsti savijeni/ispruženi
  - udaljenosti između prstiju
  - kutovi između segmenata prstiju

Samostalni zadatak 2:

Ručno definirana pravila za nove geste

- Testirati logiku na slikama ili feed-u s kamere
- Predati:
  - Kod u kojem su implementirali dvije vlastite geste
  - Screenshot gdje se vidi prepoznata vlastita gesta
  - Kratko objašnjenje pravila (“prsti X i Y su ispruženi, ostali savijeni…”)

Samostalni zadatak 3:

Klasifikator (ML) za dvije nove geste

- Odabrati dvije geste (npr. fist vs open_palm)
- Snimiti 10–20 frameova landmarka za svaku gestu:
  - izvući koordinate ključnih točaka preko MediaPipe
  - spremiti ih u CSV (ili listu u Pythonu)
  - dodati labelu (npr. “0” za fist, “1” za open_palm”)
- Napraviti najjednostavniji ML model, npr.:
  - KNN (scikit-learn)
  - Ili LDA
  - ili Logistic Regression
  - ili osnovni SVM

Samostalni zadatak 3:

Klasifikator (ML) za dvije nove geste

- Treniran model integrirati u real-time loop:
  - frame → landmarks → vektor → model.predict → ispis geste
- Predati:
  - CSV dataset (ili python dict/list) s landmarkima i labelama
  - Python kod ML modela (trening + predikcija)
  - Screenshot real-time prikaza gdje vaš model prepoznaje geste

Zadatak za predaju 2

Body pose estimation – Z2

- Ovaj dio domaće zadaće uključuje četiri zadatka:
  - Usporedba ključnih točaka (MediaPipe vs. PennAction) – PDF
  - Izračun kuta zglobova – PDF
  - Izračun nagiba tijela – PDF
  - Live tracking i izračun brzine ruke – TXT + PY datoteke
- Način predaje:
  - Zadatke 1–3 predati u jednom PDF dokumentu
  - Zadatak 4 predati kao TXT + PY datoteke
- Maksimalni broj bodova: **4 boda**

Head pose estimation – Z2

- Ovaj dio domaće zadaće uključuje 3 zadatka:
  - Vizualna interpretacija BIWI Head Pose podataka – PDF
  - Klasifikacija smjera pogleda – PDF
  - MediaPipe vs. BIWI: Usporedba orijentacije glave – PDF

- Način predaje:
  - Zadatke 1–3 predati u jednom PDFdokumentu

- Maksimalni broj bodova: **3 boda**

Gesture recognition– Z2

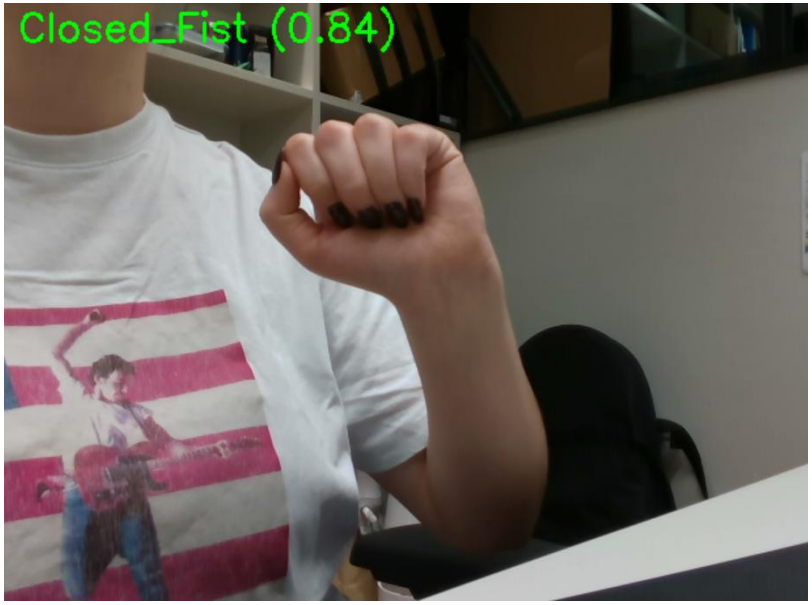
- Ovaj dio domaće zadaće uključuje 3 zadatka:
  - Gesture Recognition demo - PDF
  - Ručno definirana pravila za nove geste - PY datoteka + PDF
  - Klasifikator (ML) za dvije nove geste - PY datoteka + CSV + PDF

- Način predaje:
  - Zadatke 1–3 predati u jednom PDF dokumentu
  - Dodatno predati PY i CSV datoteke
- Maksimalni broj bodova: **3 boda**

UPUTA ZA PREDAJU:

- Kako bi pravilno predali zadaću potrebno je priložiti:
  - Izvješće – PDF - **JEDNA!** datoteka unutar koje će biti priložene sve tablice i screenshot prikazi rješenja (prije svakog prikaza navesti o kojem zadatku je riječ), npr:

    Gesture Recognition demo – Closed fist



UPUTA ZA PREDAJU:

- Kako bi pravilno predali zadaću potrebno je priložiti:
  - Sve PY datoteke za koje u zadacima **eksplicitno piše** da ih je **potrebno priložiti**
  - TXT, CSV (ili druga forma) zapis koji sadrži potrebne podatke – također navedeno u zadacima

- Rok predaje: 11.01.2026., 23:59